

Data Drift Reporter

Case Studies & Project Analysis

AI Prototype Challenge — Infinite Solutions Round 3
Automated Data Quality Monitoring with AI Narration

Team Details:
Team Name: Drift Hunters

SI.NO	Name	Roll Number	College
1	YAMA AKSHAY KUMAR	23701A3102	Annamacharya Institute of technology & sciences
2	YERRABAPU SOUKYA	23701A30C5	Annamacharya Institute of technology & sciences
3	YERIKALAREDDYGARI ARCHANA	23701A3007	Annamacharya Institute of technology & sciences

1. Project Overview

Data Drift Reporter is a full-stack AI-powered prototype that automatically monitors data quality and detects drift in business datasets. Users upload CSV snapshots of their tables, and the system computes statistics, stores historical snapshots, compares trends over time, and generates AI-narrated reports explaining what changed and why it matters.

1.1 Core Problem Statement

In real-world data pipelines, data silently degrades over time — columns accumulate null values, distributions shift, row counts drop unexpectedly. Without automated monitoring, these changes go undetected until they cause downstream failures. This project solves that by making drift visible, measurable, and explainable.

1.2 Tech Stack Summary

Component	Technology	Purpose
Backend	Python Flask	API routes, orchestration, file handling
Database	SQLite + SQLAlchemy	Stores datasets, snapshots, drift reports
Data Processing	Pandas + NumPy	Statistics computation, drift calculation
Frontend	HTML + Bootstrap 5 + Chart.js	Dashboard, upload UI, trend charts
AI Layer	Claude API (Anthropic)	Business-friendly drift narrative generation
Reports	fpdf2	PDF and CSV report export
Testing	pytest	8 happy-path tests for drift engine

1.3 AI Capability Demonstrated

- LLM Structured Output: Drift events are passed as structured JSON to the LLM, which returns a business-friendly narrative with a recommendation.
- Agent Loop: The app follows a clear read -> analyze -> call LLM -> validate -> save flow on every snapshot generation.
- Offline Fallback: If no API key is set, a deterministic rule-based narrator generates the same output structure automatically.

Case Study 1: E-Commerce Orders Dataset

Detecting Revenue Drop and Data Quality Degradation

Background

An e-commerce business tracks weekly order data in a CSV. The dataset contains `order_id`, `customer_email`, `order_amount`, `quantity`, and `region`. The team uploads two weekly snapshots — Week 1 (20 orders) and Week 2 (17 orders) — to detect whether business performance or data quality has changed.

Input Data: Week 1 Snapshot

order_id	customer_email	order_amount	quantity	region
1001	alice@example.com	120.50	2	East
1002	bob@example.com	85.00	1	West
1003	carol@example.com	210.75	3	East
...	...	...	...	...
1020	tina@example.com	160.00	2	South

Week 1 Stats: 20 rows | 0 null values | Avg order_amount: \$154.62 | All emails present

Input Data: Week 2 Snapshot

order_id	customer_email	order_amount	quantity	region
1021	(null)	95.00	2	East
1022	bob@example.com	70.00	1	West
1023	(null)	150.00	3	East
...	...	...	...	...
1037	quinn@example.com	115.00	2	North

Week 2 Stats: 17 rows | 5 null emails (29.4%) | Avg order_amount: \$126.18 | 3 rows missing

Drift Detection Results

Metric	Week 1	Week 2	Change	Drift Signal
Row Count	20	17	-15.0%	High
Null Rate (overall)	0.00%	23.53%	+23.53 pp	High
Avg order_amount	\$154.62	\$126.18	-18.3%	High

Metric	Week 1	Week 2	Change	Drift Signal
customer_email nulls	0 (0%)	5 (29.4%)	+29.4 pp	High
Drift Score	-	-	21.5 / 100	HIGH

AI-Generated Narrative

AI Report: Average order_amount decreased by 18.3% (from \$154.62 to \$126.18). 'customer_email' null rate increased from 0.00% to 29.41%. Row count decreased by 15.0%, indicating reduced data volume. Recommendation: Investigate the upstream data pipeline for Week 2 — the combination of missing emails and revenue drop suggests either a CRM integration failure or a data extraction error, not a genuine business decline.

Business Interpretation

- The 15% row count drop means 3 orders from the previous week are entirely missing in Week 2 — likely a pipeline or extraction issue.
- 29.4% of customer_email values are null in Week 2, a sharp rise from 0%. This suggests the CRM or email capture system had an outage or integration break.
- The average order amount drop of 18.3% correlates with the missing rows — the missing orders may have been higher-value transactions.
- Drift Level: HIGH — immediate investigation of the ETL pipeline is warranted before this data is used in reporting or ML models.

What the Project Demonstrated

- End-to-end flow: CSV upload → stats computation → drift scoring → AI narration → PDF report download.
- LLM structured output: Drift events passed as JSON, narrative returned as plain English with a clear recommendation.
- The offline fallback rule-based narrator produced an identical structure when the API key was not set.

Case Study 2: Customer Registrations Dataset

Monitoring New User Cohort Quality Over Two Onboarding Batches

Background

A SaaS product tracks customer registrations. The dataset contains customer_id, name, email, age, signup_date, and country. Batch 1 covers the first 10 customers (January-February signups). Batch 2 covers the next 8 customers (March-April signups). The team wants to monitor whether data completeness and customer profile characteristics are consistent across onboarding batches.

Input Data Comparison

Field	Batch 1 (Week 1)	Batch 2 (Week 2)	Change
Rows	10	8	-20.0%
Email nulls	0 (0%)	3 (37.5%)	+37.5 pp
Avg age	35.7 years	54.0 years	+51.3%
Countries	3 (USA, UK, Canada)	4 (+ India)	New region added
Null rate overall	0.0%	12.5%	+12.5 pp

Drift Detection Results

Metric	Value	Drift Level
Row Count Change	-20.0%	Medium
Overall Null Rate Change	0% → 12.5%	Medium
Avg Age Shift	+51.3%	High
Email Column Null Change	0% → 37.5%	High
Overall Drift Score	18.7 / 100	HIGH

AI-Generated Narrative

AI Report: The customer_email null rate rose sharply from 0% to 37.5% in Batch 2 — 3 of 8 new customers are missing email addresses, which will break email onboarding flows. The average customer age increased by 51.3% (35.7 → 54.0 years), suggesting this batch represents a different demographic segment. Row count declined by 20%, consistent with a smaller signup cohort. Recommendation: Fix the email capture form before the next batch — missing emails will block automated onboarding sequences and skew engagement analytics.

Business Interpretation

- 37.5% email null rate in Batch 2 is a critical data quality issue — email is the primary communication channel for SaaS onboarding.
- The average age jump from 35.7 to 54.0 years is a significant demographic shift. This could indicate a successful enterprise or senior-market campaign, or a data entry issue in the age field.
- A new country (India) appears in Batch 2, suggesting international expansion is underway — this should be reflected in localization and support coverage.
- Drift Score: HIGH — the email null rate alone is enough to halt automated onboarding for Batch 2 customers.

What the Project Demonstrated

- Per-column drift tracking: The system identified the specific column (customer_email) causing the high drift score, not just the overall score.
 - Multi-signal reasoning: The AI combined null rate, demographic shift, and row count into a single coherent narrative.
 - Actionable output: The recommendation was specific enough for a product manager to act on immediately.
-

Case Study 3: Low Drift Baseline Validation

Confirming Stable Data When No Real Change Has Occurred

Background

A critical requirement of any drift detection system is that it does not raise false alarms when data is stable. This case study validates the system's behaviour when the same dataset is compared to itself — the expected result is a drift score near zero and a 'Low' classification.

Test Scenario

Scenario: Upload orders_week1.csv as the baseline. Generate a second snapshot from the same file without any changes. The system should detect no meaningful drift.

Results

Metric	Snapshot 1	Snapshot 2	Change	Drift Signal
Row Count	20	20	0.0%	None
Overall Null Rate	0.00%	0.00%	0.0 pp	None
Avg order_amount	\$154.62	\$154.62	0.0%	None
Drift Score	-	-	0.0 / 100	LOW

AI Report: No significant changes detected compared to the previous snapshot. All column statistics are within normal variance. Recommendation: No action required — data quality is stable.

Why This Matters

- A drift detector that fires on stable data is worse than no detector — it causes alert fatigue and erodes trust.
- This test confirms the system correctly returns Drift Level: Low for identical snapshots, validating the scoring formula.
- This scenario is also covered by the automated test: `test_identical_snapshots_give_low_drift` in `tests/test_basic.py`.

4. System Architecture & Data Flow

End-to-End Flow

Step	Component	What Happens
1. Upload	app.py → Flask route /upload	User uploads CSV. File saved to uploads/. First snapshot computed immediately.
2. Stats	drift_engine.compute_snapshot_stats()	Row count, null rate, per-column mean/median/min/max/std computed via Pandas.
3. Store	models.py → SQLAlchemy → SQLite	Snapshot saved to DB with full stats_json blob for future comparison.
4. Compare	drift_engine.compare_snapshots()	New snapshot vs previous: row/null/mean % changes calculated. Drift events generated.
5. Score	drift_engine.classify_drift()	Drift score 0-100 mapped to Low (<5), Medium (5-15), High (>15).
6. Narrate	narrator.py → Claude API	Drift events sent as JSON to LLM. Business narrative + recommendation returned.
7. Report	report_generator.py → fpdf2	PDF and CSV reports generated and made available for download.
8. Display	templates/ → Chart.js	Dashboard shows drift score, trend charts, AI narrative, snapshot history.

Drift Score Formula

The drift score is the mean of four components:

- Row count % change between snapshots
- Overall null rate % change between snapshots
- Overall mean % change between snapshots
- Maximum per-column drift signal (largest of null rate jump and mean shift across all columns)

Formula: `drift_score = mean(row_change_pct, null_change_pct, mean_change_pct, max_column_drift)`

5. Test Cases

Automated Tests (tests/test_basic.py)

Test Name	What It Validates	Expected Result
test_compute_stats_returns_required_keys	Stats dict has row_count, overall_null_rate, overall_mean	PASS
test_compute_stats_row_count	row_count matches DataFrame length	PASS
test_compute_stats_null_rate_between_0_and_1	null_rate is a float between 0 and 1	PASS
test_compare_snapshots_returns_score_and_level	Drift result has drift_score and drift_level	PASS
test_drift_level_valid_values	drift_level is one of Low, Medium, High	PASS
test_drift_score_non_negative	drift_score is ≥ 0	PASS
test_identical_snapshots_give_low_drift	Same snapshot vs itself = Low drift	PASS
test_classify_drift_thresholds	Score 3 = Low, 10 = Medium, 20 = High	PASS

Result: 8/8 tests passed. Run with: `python -m pytest tests/test_basic.py -v`

6. Assumptions & Limitations

Assumptions

- Each uploaded CSV represents one point-in-time snapshot of a single business table.
- Drift is detected by comparing the most recent snapshot to the immediately preceding one (not a rolling average).
- The LLM narration is supplementary — the numeric drift score is the authoritative signal.
- Sample data uses small datasets (8-20 rows) appropriate for a prototype demonstration.

Current Limitations

- No authentication: The upload endpoint is open. Production deployment requires auth middleware.
- Single-file datasets only: Each dataset maps to one CSV. Multi-table joins are not supported.

- No scheduled jobs: Snapshot generation is manual (click button). Production would use APScheduler or cron.
- SQLite only: Not suitable for multi-user concurrent access. Replace with PostgreSQL for production.
- LLM dependency: AI narration requires an Anthropic API key. The fallback is rule-based, not ML-powered.

Suggested Improvements

- Add column-type-aware drift detection (e.g., chi-squared test for categorical columns).
 - Support scheduled auto-snapshot generation via APScheduler.
 - Add email/Slack alerting when drift level reaches High.
 - Add user authentication and multi-tenant dataset isolation.
 - Replace the rule-based fallback with a locally-hosted open-source LLM (e.g., Ollama + Llama 3).
-

7. Summary

Data Drift Reporter successfully demonstrates a working end-to-end AI prototype that solves a real data engineering problem. Across three case studies, the system correctly identified high-severity drift in an e-commerce orders dataset (revenue drop + email data loss), detected a demographic and quality shift in a customer registrations dataset, and validated stable behaviour with a near-zero drift score when no real change occurred.

The AI narration layer added genuine value by translating numeric drift scores into actionable business recommendations — bridging the gap between data engineering signals and business decision-making. All 8 automated tests passed, the app runs fully offline without an API key, and the codebase follows the recommended folder structure and submission checklist from the Infinite Round 3 guidelines.